ctc_asr

In-process Korean speech-to-text (fine-tuned Wav2Vec2-CTC), with near-real-time streaming.

GitHub: github.com/Aditi1612/ctc_asr_library · Model: huggingface.co/fluffy16/ctc-asr-ko · v0.1.0

What it is

A Python library that runs the Korean ASR model **directly inside your application** — no HTTP server, no calling an API endpoint. You load the model once and call a function.

- **One-shot**: give it an audio file or waveform, get the transcript back.
- **Streaming**: push microphone frames continuously; get a transcript for each utterance when the speaker pauses (voice-activity segmented).

The decode path (audio normalization, jamo normalization, Hangul composition, CTC decoding) is identical to the production API, so output matches.

Requirements

Component	Notes
Python	3.9 or newer
`torch`, `torchaudio`	Install the build that matches your machine (CUDA or CPU) *first*. These are large and hardware-specific.
GPU	Optional. CUDA gives ~25 ms per clip; CPU works fine (~0.04 s per second of audio).
Disk	~1.3 GB for the model, downloaded once and cached in `~/.cache/huggingface`.

Installation

```
# 1. install torch for your hardware (example: CUDA 12.1)
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu121
#    ...or CPU-only:
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cpu

# 2. install the library (public GitHub repo)
pip install "ctc-asr[hub] @ git+https://github.com/Aditi1612/ctc_asr_library.git"
```

Optional extras:

Extra	Adds
`[hub]`	download the model from Hugging Face Hub (recommended)
`[vad]`	`webrtcvad` — sharper speech detection than the built-in energy fallback
`[mic]`	`sounddevice` — only needed to run the microphone demo

Quick start — one-shot

```
from ctc_asr import Transcriber

asr = Transcriber(hf_repo="fluffy16/ctc-asr-ko")   # ~1.2 GB download on first run, then
cached

print(asr.transcribe("clip.wav"))                  # -> "카촉틀과 토이띠차 내뒤에 안자쩌요"

# raw waveform instead of a file (mono float32):
import soundfile as sf
wav, sr = sf.read("clip.wav", dtype="float32")
print(asr.transcribe(wav, input_sr=sr))

# full detail (raw text, CER if you pass reference_text, timing):
asr.transcribe("clip.wav", detailed=True)
```

Accepts a file path, encoded bytes (`wav/flac/ogg` , plus `mp3/m4a` when ffmpeg is present), or a numpy / torch waveform.

Streaming (near real-time)

Your app captures audio from anywhere and pushes mono float32 chunks of any length. Each time the speaker pauses, the buffered utterance is transcribed and returned. There are no partial hypotheses — a full transcript per utterance.

```
from ctc_asr import Transcriber

asr = Transcriber(hf_repo="fluffy16/ctc-asr-ko")
stream = asr.stream(on_segment=lambda s: print(f"[{s.start_time:.1f}s] {s.text}"))

for chunk in your_mic_source():                # e.g. 20-100 ms blocks, any sample rate
    for seg in stream.accept(chunk, input_sr=48000):   # resampled internally if needed
        handle(seg.text, seg.start_time, seg.end_time)

last = stream.flush()                           # finalize trailing speech when the stream
ends
```

`accept()` returns a list of finalized `Segment` objects — usually empty, one when an utterance just ended. It never blocks waiting for the microphone; the model only runs when a pause is detected.

Tuning `asr.stream(...)`

Argument	Default	Meaning
`end_silence_ms`	600	pause length that ends an utterance — this is roughly your perceived latency
`min_speech_ms`	250	utterances shorter than this are dropped as noise
`max_speech_ms`	15000	force-cut a segment that never pauses
`start_speech_frames`	3	consecutive speech frames needed to start capturing
`pre_roll_ms`	200	audio kept from just before the trigger, so onsets aren't clipped
`vad_aggressiveness`	2	webrtcvad strictness 0–3 (higher = more aggressive about calling things non-speech)

Where the model comes from

Resolution priority: explicit paths → Hugging Face repo → local sibling checkout.

Source	How
Hugging Face Hub	`Transcriber(hf_repo="fluffy16/ctc-asr-ko")` or `export CTC_HF_REPO=fluffy16/ctc-asr-ko`
Local files	`Transcriber(model_dir="/path/checkpoint", vocab_path="/path/vocab.json")` or `CTC_MODEL_DIR` / `CTC_VOCAB_PATH`
Pin a version	`Transcriber(hf_repo="fluffy16/ctc-asr-ko", hf_revision="<commit-sha>")`

> The Hugging Face model repo is currently **public** — no token needed. If it is switched to private later, each user runs `huggingface-cli login` once (or sets `HF_TOKEN`) and is added as a collaborator.

Performance (measured on the sample set)

Times are for the `transcribe()` call only, with the model already loaded.

Audio length	GPU (CUDA)	CPU
2 s	0.012 s	0.11 s
5 s	0.025 s	0.20 s
14 s	0.026 s	0.62 s
22 s	0.034 s	0.83 s

- **GPU**: ~25 ms regardless of clip length (real-time factor ≈ 0.003).
- **CPU**: ~0.04 s per second of audio, scales linearly (RTF ≈ 0.04).
- **Model load into memory**: ~0.4 s, once, at `Transcriber(...)`.
- **First download from Hugging Face**: ~1.2 GB — network-dependent, then cached permanently.

Limitations

- The model transcribes whole utterances — there is no word-by-word live output. “Real-time” means the transcript appears shortly after each pause.
- Segmenting on silence can split a word at the boundary, so a long sentence spoken without pauses transcribes slightly differently than the same audio sent as one clip.
- One `Transcriber` runs a single inference at a time. For parallelism, use one instance per worker thread / process.
- Korean output only; the model is fine-tuned for dysarthric / atypical speech.

Updating

When	Maintainer does	Users do
Library code changes	`git push` to the GitHub repo	`pip install --upgrade "git+https://github.com/Aditi1612/ctc_asr_library.git"`
Model checkpoint changes	re-run the Hugging Face upload script	nothing — next run pulls the new snapshot (unless `hf_revision` is pinned)

Troubleshooting

Symptom	Fix
`No checkpoint found`	pass `hf_repo=` / `model_dir=`, or set `CTC_HF_REPO`
`Hugging Face Hub support needs 'huggingface_hub'`	install with the `[hub]` extra
VAD backend is `energy`, detection is rough	`pip install webrtcvad`
mp3 / m4a files fail to decode	install ffmpeg on the system
CUDA out of memory	`Transcriber(force_cpu=True)`